

Chapter 9: Virtual Memory





Background

- **Virtual memory** – separation of user logical memory from physical memory.
 - Only part of the program needs to be in memory for execution.
 - Logical address space can therefore be much larger than physical address space.
 - Allows address spaces to be shared by several processes.
 - Allows for more efficient process creation.
- Virtual memory can be implemented via:
 - Demand paging
 - Demand segmentation





Virtual Memory That is Larger Than Physical Memory

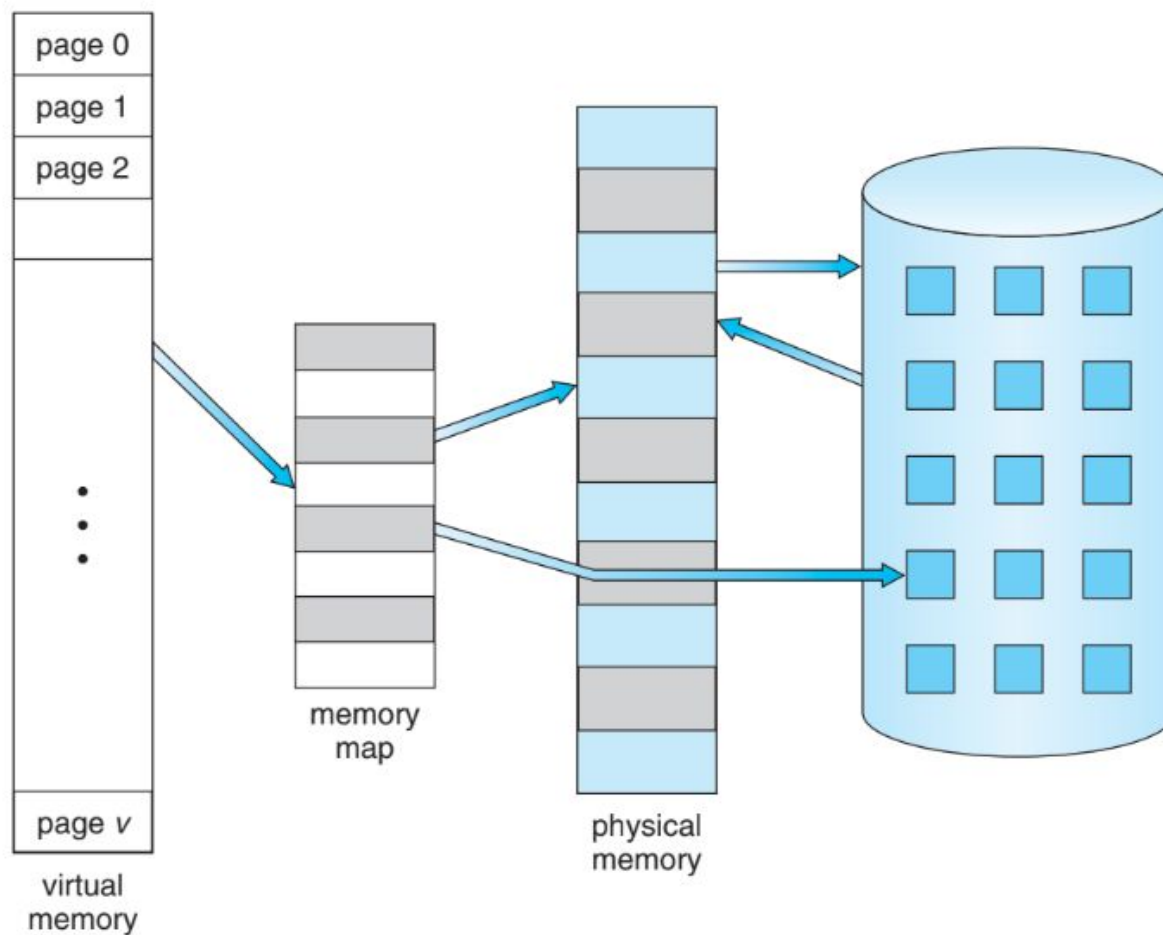
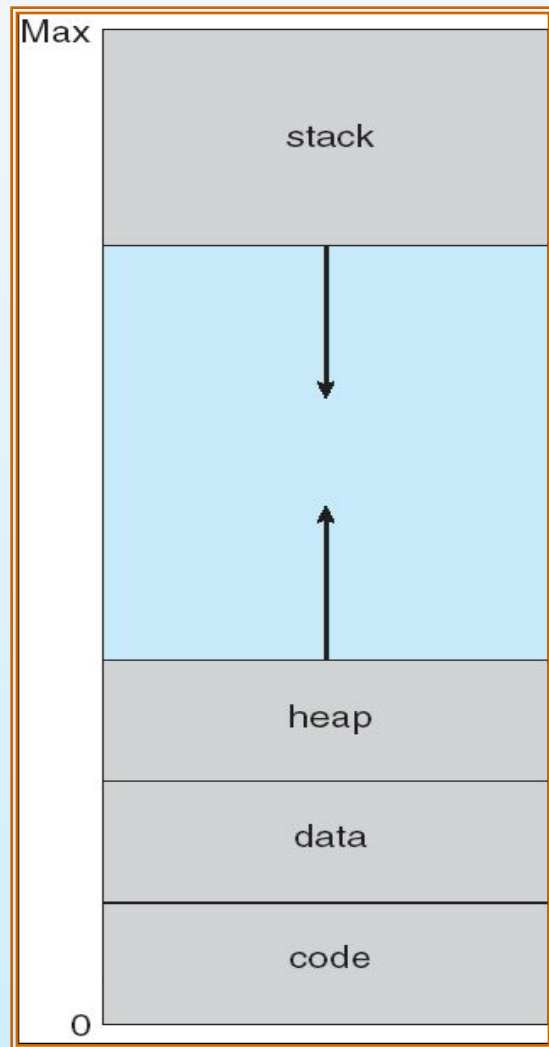


Figure 9.1 Diagram showing virtual memory that is larger than physical memory.





Virtual-address Space





Shared Library Using Virtual Memory

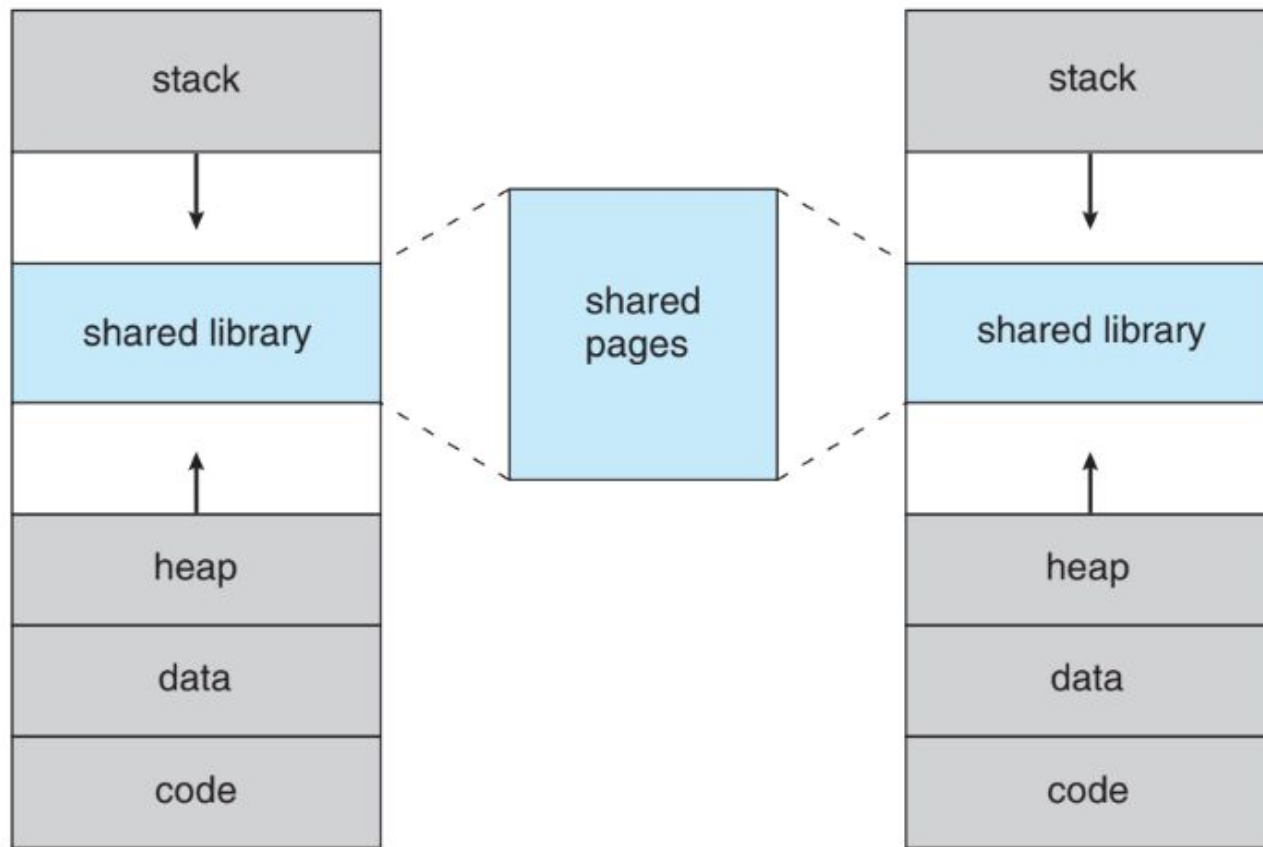


Figure 9.3 Shared library using virtual memory.





Demand Paging

- Bring a page into memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
- Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory





Demand Paging

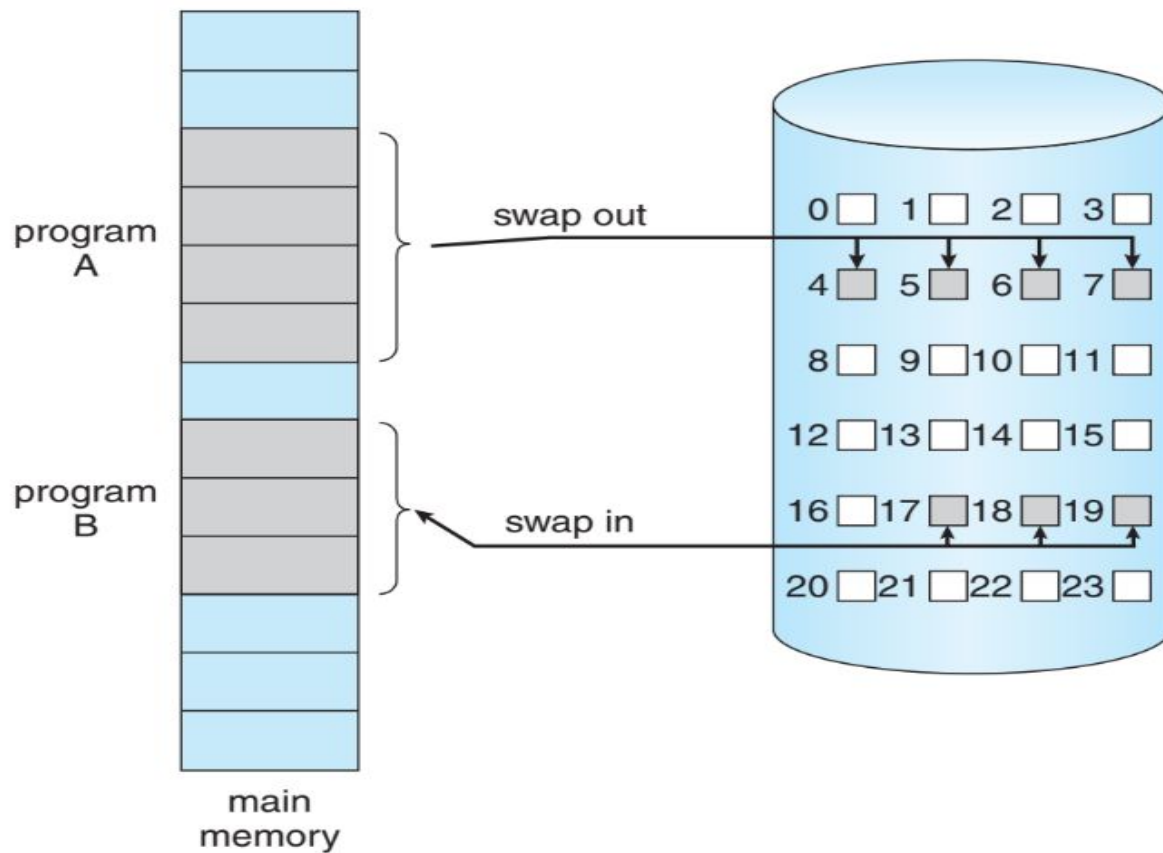


Figure 9.4 Transfer of a paged memory to contiguous disk space.





Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (1 $\Rightarrow$ in-memory, 0 $\Rightarrow$ not-in-memory)
- Initially valid–invalid but is set to 0 on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	1
	1
	1
	1
	0
☐	
	0
	0

page table





Page Table When Some Pages Are Not in Main Memory

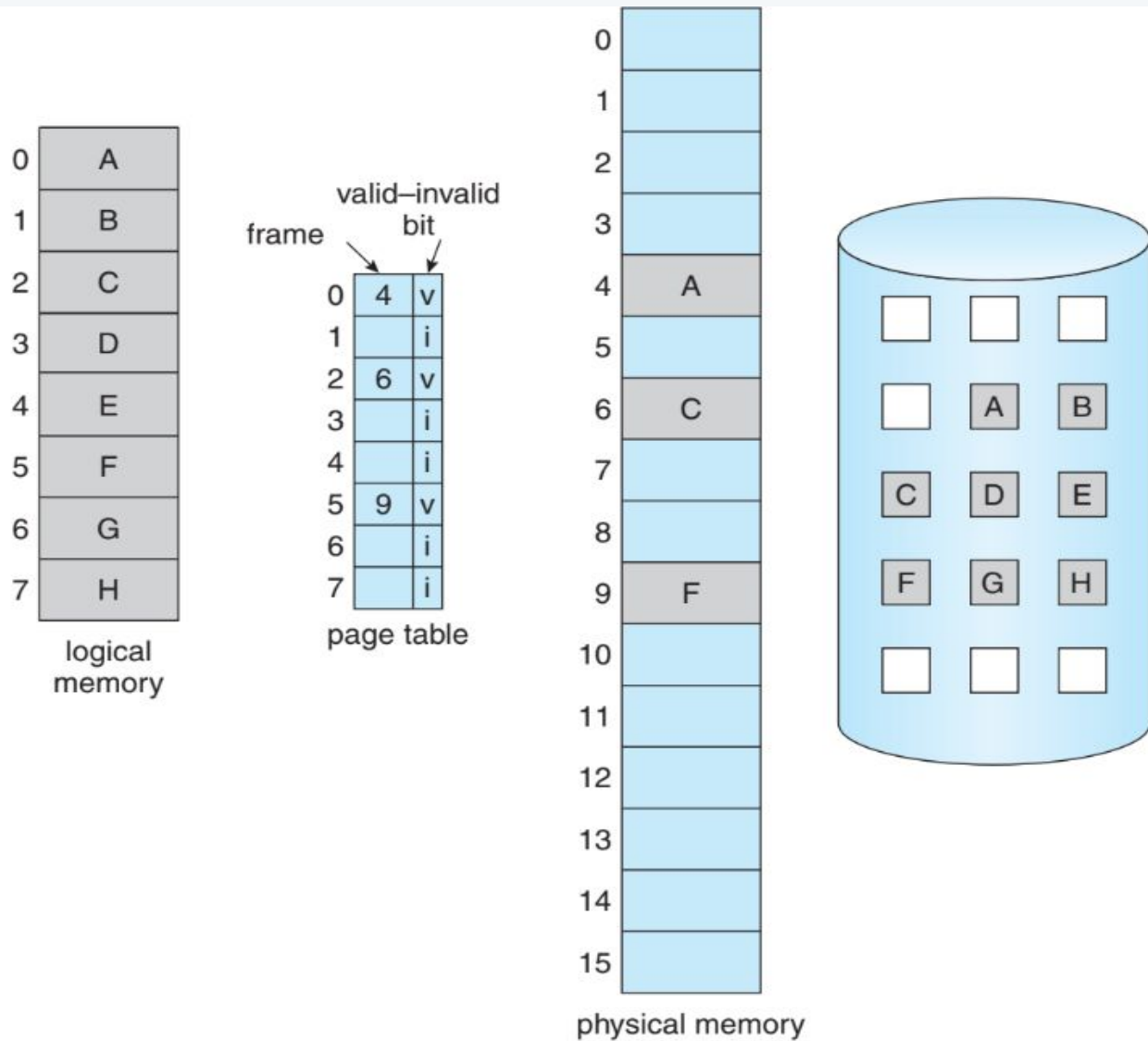


Figure 9.5 Page table when some pages are not in main memory.





Page Fault

- If there is ever a reference to a page, first reference will trap to OS $\Rightarrow$ **page fault**
- OS looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort.
 - Just not in memory.
- Find empty frame.
- Load page from disk into frame.
- Reset tables, validation bit = 1.
- Restart instruction that caused page fault





Steps in Handling a Page Fault

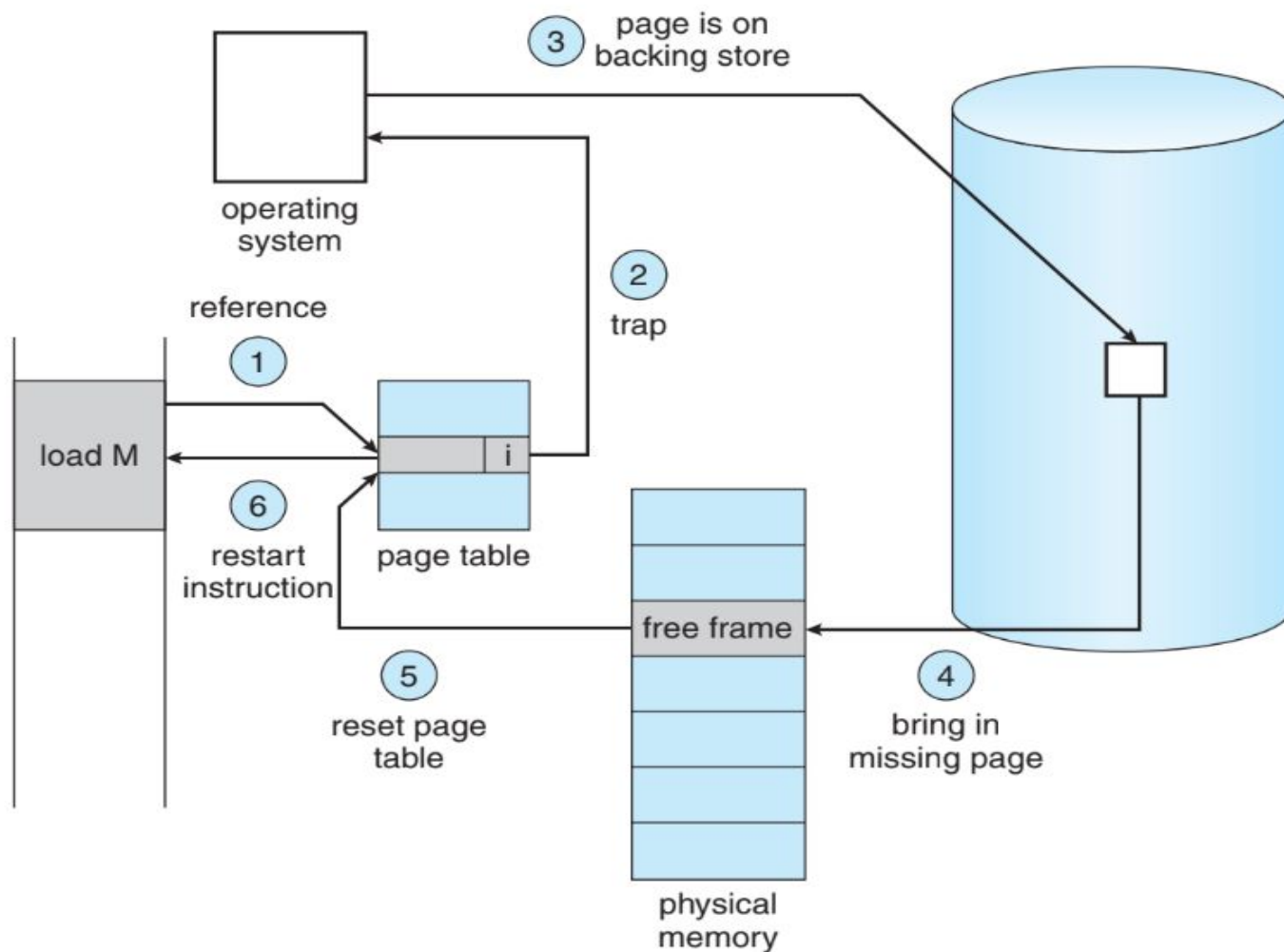


Figure 9.6 Steps in handling a page fault.





Page fault handling

The procedure for handling page fault is straightforward:

1. We check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.
2. If the reference was invalid, we terminate the process. If it was valid but we have not yet brought in that page, we now page it in.
3. We find a free frame (by taking one from the free-frame list, for example).
4. We schedule a disk operation to read the desired page into the newly allocated frame.
5. When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.





What happens if there is no free frame?

- **Page replacement** – find some page in memory, but not really in use, swap it out
 - algorithm
 - performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times





Need For Page Replacement

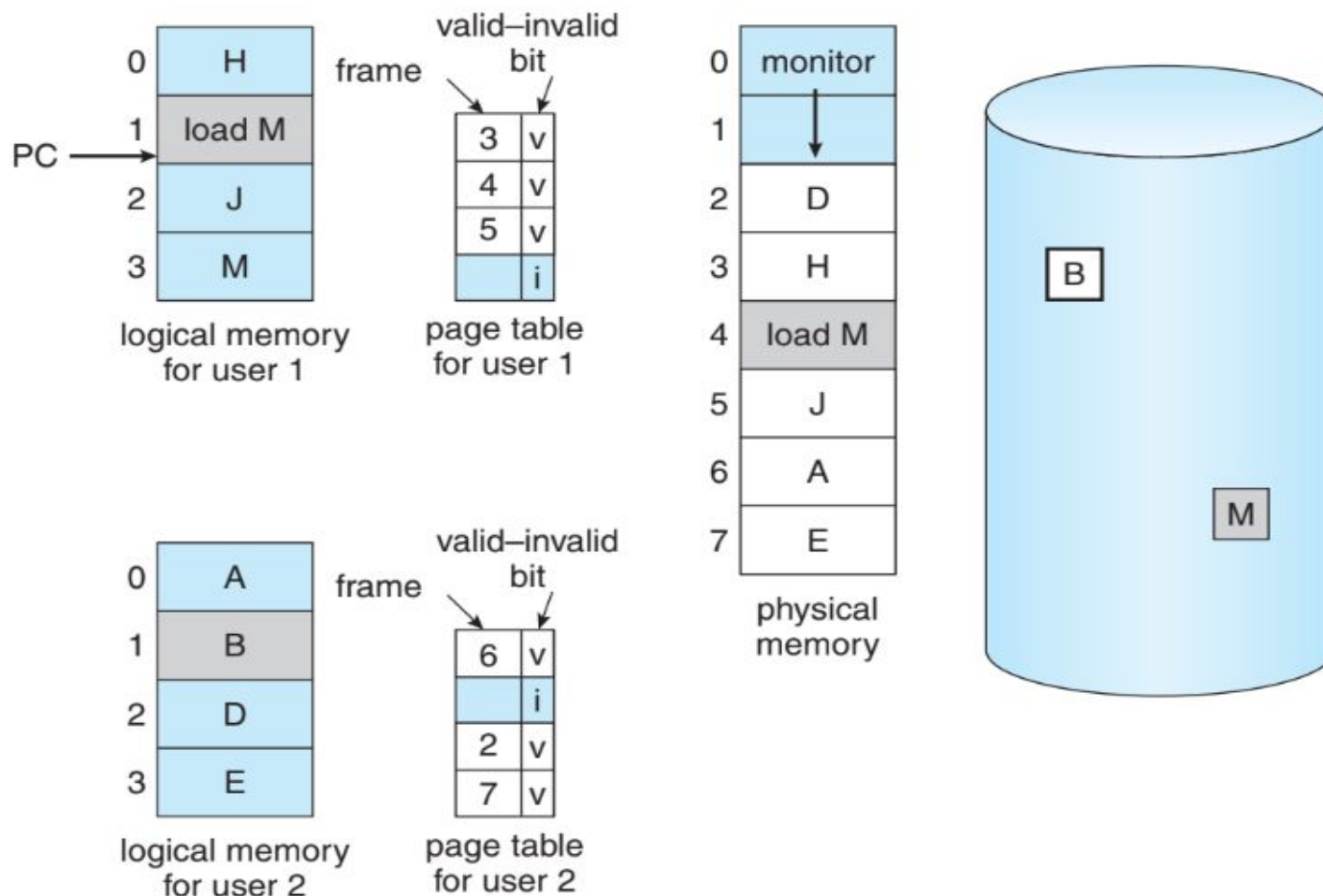


Figure 9.9 Need for page replacement.





Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use a page replacement algorithm to select a **victim** frame
3. Read the desired page into the (newly) free frame. Update the page and frame tables.
4. Restart the process





Page Replacement

9.4 Page Replacement

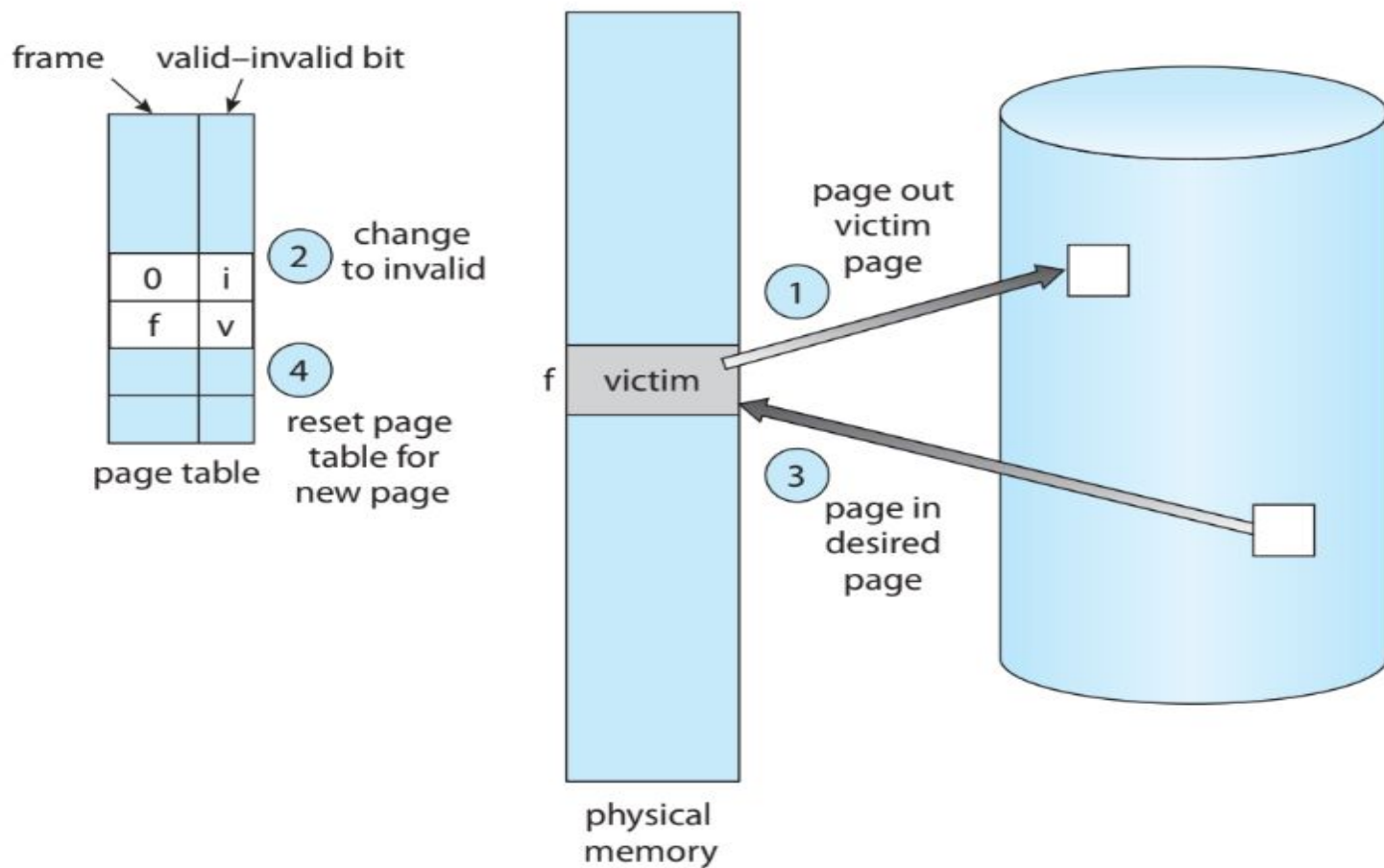


Figure 9.10 Page replacement.





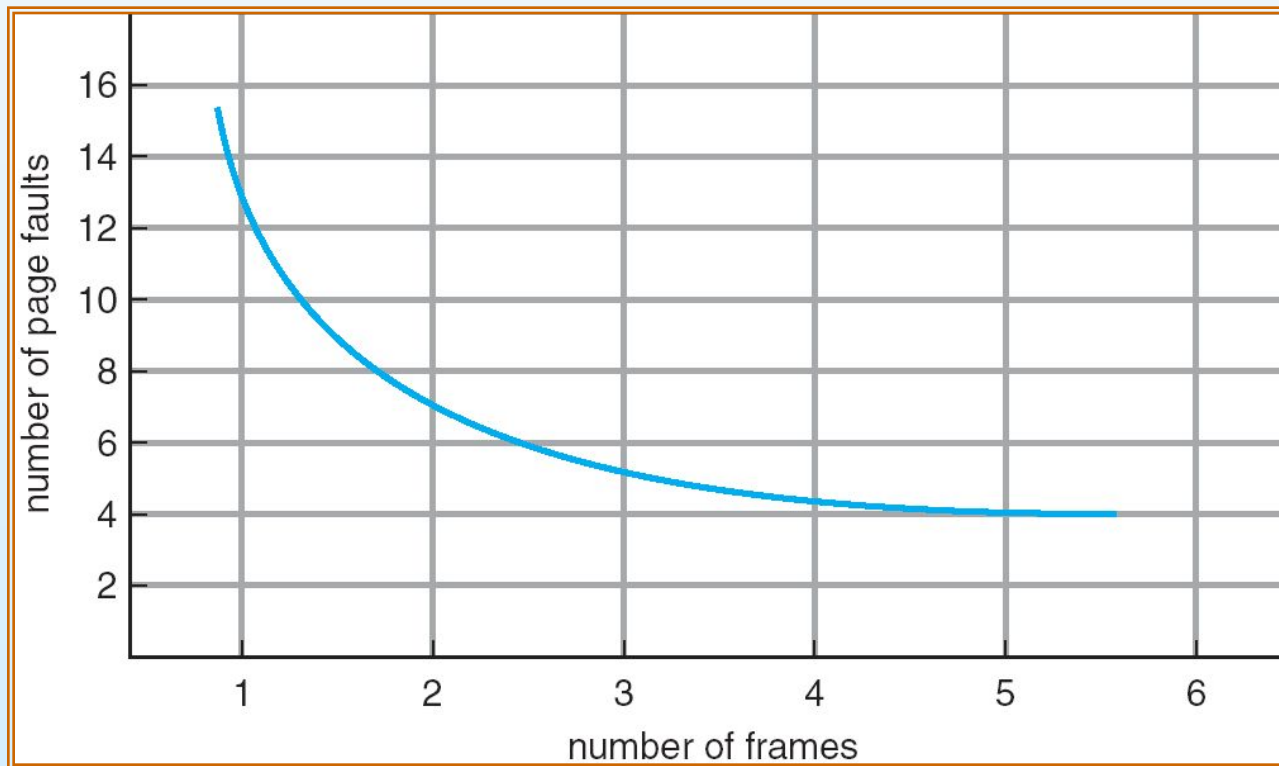
Page Replacement

- Use **modify (dirty) bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory
- Solve two problems in demand paging implementation:
 - **Frame-allocation algorithm** – how many frames to allocate to each process
 - **Page-replacement algorithm** – select frames to be replaced





Graph of Page Faults Versus The Number of Frames





Page Replacement Algorithms

- Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (**reference string**) and computing the number of page faults on that string
- In all our examples, the reference string is
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5





First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)

1	1	4	5	
2	2	1	3	9 page faults
3	3	2	4	

- 4 frames

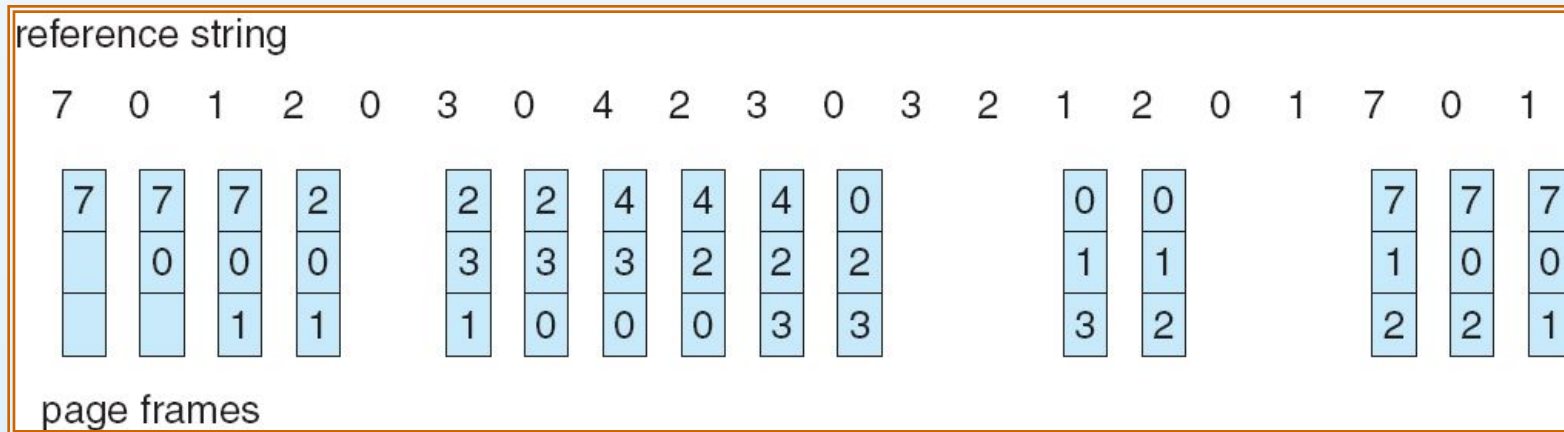
1	1	5	4	
2	2	1	5	10 page faults
3	3	2		
4	4	3		

- FIFO Replacement – Belady’s Anomaly



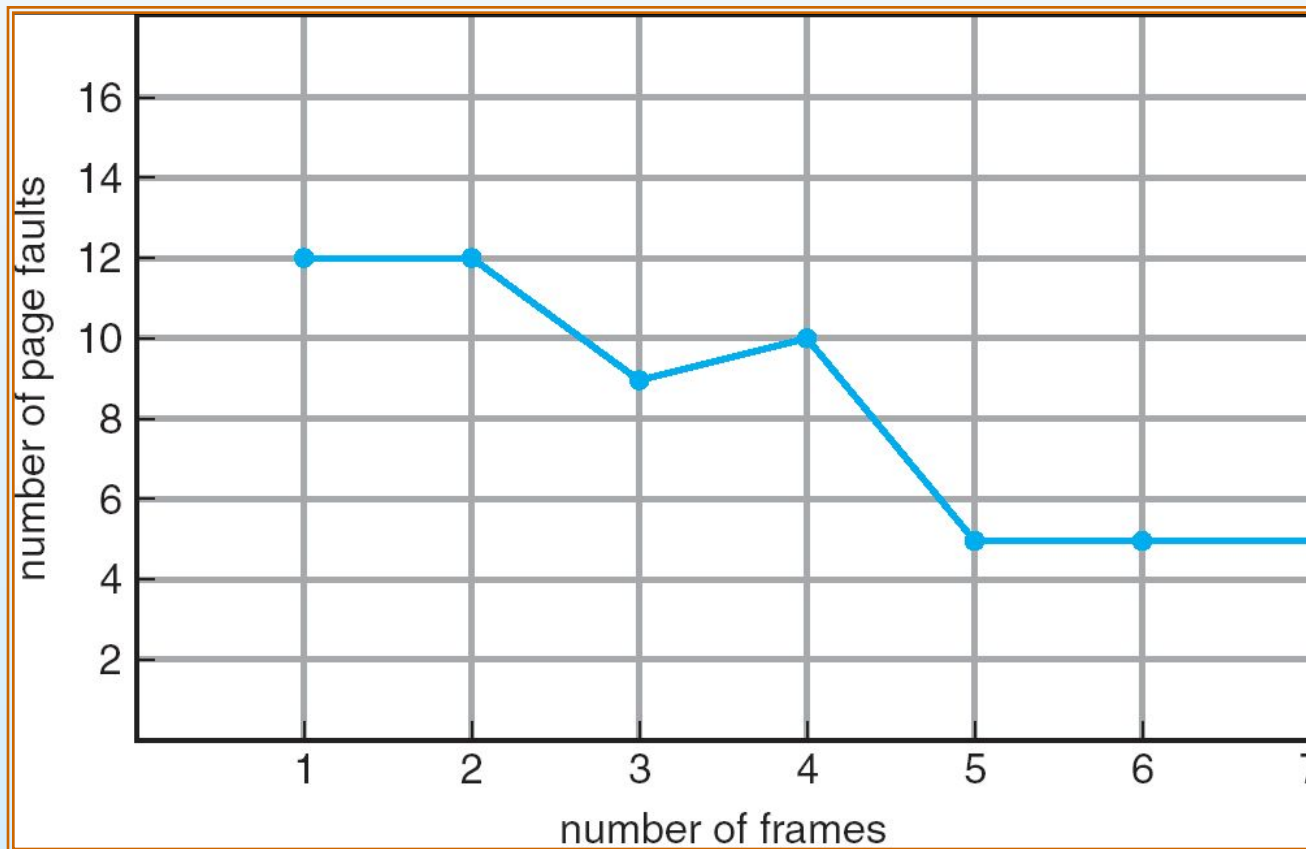


FIFO Page Replacement





FIFO Illustrating Belady's Anomaly





Optimal Algorithm

- Replace page that will not be used for longest period of time
- 4 frames example

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	4
2	
3	
4	5

6 page faults

- How do you know this?
- Used for measuring how well your algorithm performs





Optimal Page Replacement

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2		2						7
	0	0	0		0		0		0		0						0
		1	1		3		3		3		1						1

page frames





Least Recently Used (LRU) Algorithm

- LRU replaces page that has not been used for the longest time
- Use the recent past to predict the future
- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

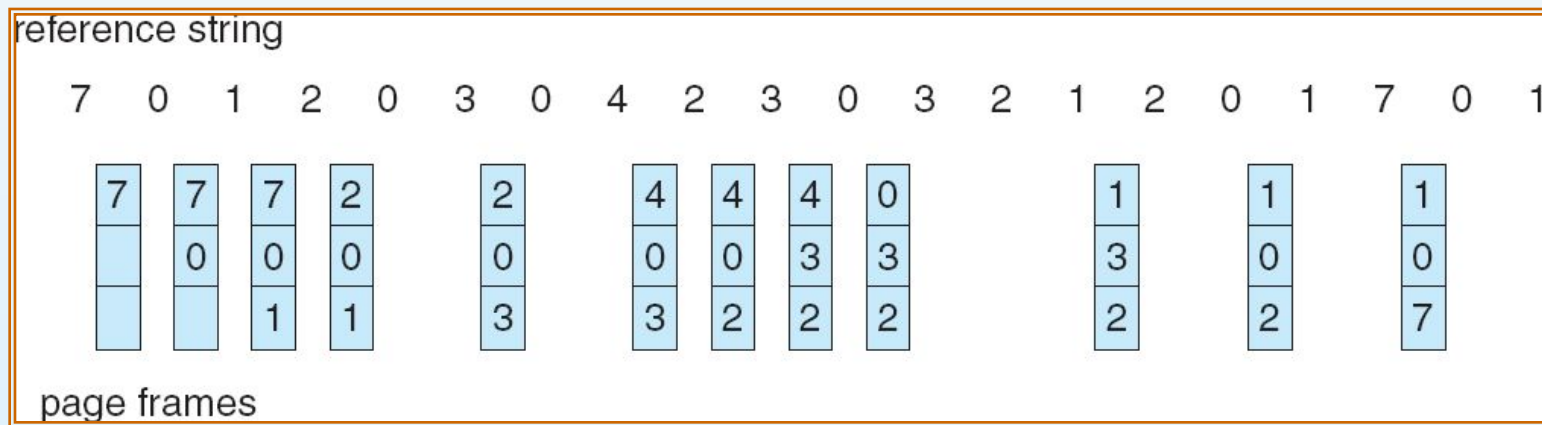
1	5
2	
3	5 4
4	3

8 page faults





LRU Page Replacement





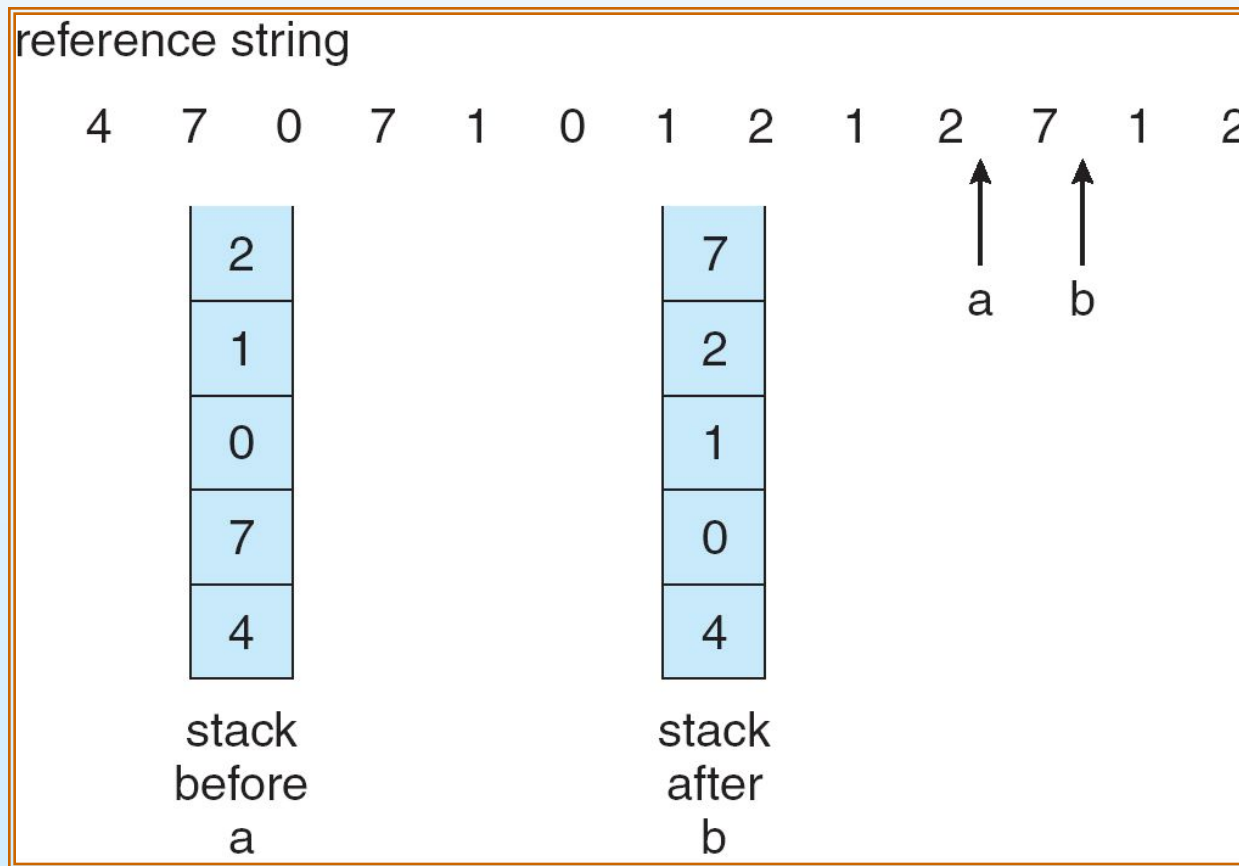
LRU Algorithm (Cont.)

- **Counter** implementation
 - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be replaced, look at the counters to determine which has the oldest time-of-access
- **Stack** implementation – keep a stack of page numbers in a double link form:
 - Page referenced -> move it to the top of stack
 - 4 bottom of stack will be the LRU page
 - No search for replacement





Use Of A Stack to Record The Most Recent Page References





Thrashing

- If a process does not have “enough” frames, the **page-fault rate is very high**. This leads to:
 - low CPU utilization
 - operating system thinks that it needs to increase the degree of multiprogramming
 - another process added to the system
- **Thrashing** $\equiv$ a process is busy swapping pages in and out



End of Chapter 9

